Université Paris Sorbonne Nord
Institut Galilée

Master 1.
Distributed Java Programming
TP1: Inheritance and polymorphism

1 Java Environments

Several environments exist for developing Java programs. It is critical to use a *good* version: the same one for compiling and executing a program (essentially when classes are dynamically loaded). The available Java distribution at Institut Galilée is java 1.8.0 (`/usr/bin/java`) from SUN/Oracle Java SE. There exists also a GNU open source in some PCs, not fully compliant with Oracle/Sun Java.

1.1 Eclipse

Eclipse is an integrated development environment (IDE), free, extensible and multi-language, able to plug tools for any programming language. Eclipse is mainly developed in Java (with the graphic library SWT from IBM). The IDE Eclipse is developed by means of plug-ins (compliant with the OSGi norm). See `www.eclipse.org`.
A **project** in Eclipse is created via: `File -> New -> Java Project`
The window *Project Explorer* allows you to travel through projects and source files
`Window -> Show view -> Project Explorer`
A **workspace** contains the whole set of projects and files. The workspace is a folder in your home directory, defined when launching Eclipse. You can retrieve the path by `File -> Switch Workspace`.
Each project is a sub-directory containing a folder `bin` for the executable files, and a folder `src` for source files.
In order to create a new file/class, use the option `New -> Class`. If the new class contains a method `main`, then it can be executed via `run -> run`.

1.2 Netbeans Environment

NetBeans is also an open source IDE for Java, developed by Sun. It can also support other languages such as Python, C, C++, XML, HTML.
A **project** can be created via `File -> New Project -> Java -> Java Application`

2 Classes and inheritance

For the following questions, think of what should be the output, then run the code and compare with your answer.

```
class Exo1{
    private int val =0;
    public static void affichePlus ( int a ) {
        a ++;
        System.out.println ( a );
    }

    public static void affichePlus ( Exo1 a ) {
        a.val ++;
        System.out.println ( a.val );
    }

    public static void main ( String [] args ) {
        Exo1 O1 = new Exo1 ();
        Exo1 O2 = new Exo1 ();
        Exo1.affichePlus ( O1.val );
        Exo1.affichePlus ( O1.val );
        Exo1.affichePlus ( O1 );
        Exo1.affichePlus ( O1 );
        Exo1.affichePlus ( O2 );
        Exo1.affichePlus ( O2 );

        if ( O1 == O2 )
            System.out.println ( " Equals " );
        else
            System.out.println ( " Different " );
    }
}
```

```
class A {
    public String f ( B obj ) {
        return ( " A and B " );
    }

    public String f ( A obj ) {
        return ( " A and A " );
    }
}

class B extends A {
    public String f ( B obj ) {
        return ( " B and B " );
    }

    public String f ( A obj ) {
        return ( " B and A " );
    }
}

class Exo2 {
    public static void main ( String [] args ){
        A a1 = new A ();
        A a2 = new B ();
        B b = new B ();
        System.out.println ( a1.f ( a1 ));
        System.out.println ( a1.f ( a2 ));
        System.out.println ( a2.f ( a1 ));
        System.out.println ( a2.f ( a2 ));
        System.out.println ( a2.f ( b ));
        System.out.println ( b.f  ( a2 ));
    }
}
```

4 Polymorphism

```
class A {
    public String f ( D obj ) {
        return ( " A and D " );
    }

    public String f ( A obj ) {
        return ( " A and A " );
    }
}

class B extends A {
    public String f ( B obj ) {
        return ( " B and B " );
    }

    public String f ( A obj ) {
        return ( " B and A " );
    }
}
class C extends B {
}

class D extends B {
}

class Exo3 {
    public static void main ( String [] args ){
        A a1 = new A ();
        A a2 = new B ();
        B b = new B ();
        C c = new C ();
        D d = new D ();
        System.out.println ( a1.f ( b ));
        System.out.println ( a1.f ( c ));
        System.out.println ( a1.f ( d ));
        System.out.println ( a2.f ( b ));
        System.out.println ( a2.f ( c ));
        System.out.println ( a2.f ( d ));
        System.out.println ( b. f ( b ));
        System.out.println ( b. f ( c ));
        System.out.println ( b. f ( d ));
    }
}
```

Was it confusing? Consider a call `o.m(p)`:

- Overridden non-static methods are selected at **run time** based on the *dynamic type* of `o`.
- The *static type* of `p` is used to determine the method to be executed (including overloaded methods).

See more at `https://docs.oracle.com/javase/tutorial/java/IandI/override.html`.

```
class A {
    int i ;
    int f () { return i ; }
    static String g () { return ( " A " ); }
    String h () { return g (); }
}

class B extends A {
    int i =2;
    int f () { return -i ; }
    static String g () { return ( " B " ); }
    String h () { return g (); }
}

class Exo4 {
    public static void main ( String [] args ) {
        B b = new B ();
        System.out.println ( b.i );
        System.out.println ( b.f ());
        System.out.println ( b.g ());
        System.out.println ( b.h ());
        A a = b;
        System.out.println ( a.i );
        System.out.println ( a.f ());
        System.out.println ( a.g ());
        System.out.println ( a.h ());
    }
}
```

Hiding: using a variable in the subclass with the same name of a variable in the superclass; or defining a static method in the subclass with the same signature of the superclass. This is not a good style of programming and better to avoid it!

6 Game

The goal is to implement a character system for a small combat game. The game contains several types of characters that share some common properties but have different behaviors.
The program must make it possible to manipulate characters of different types through a common interface, without knowing their concrete type when performing an action.

Characters

Each character has:

- a name;
- a current number of health points;
- a maximum number of health points, fixed when the character is created;
- an inventory containing items.

Health points are always between 0 and the maximum number of health points. A character whose health points are zero is dead and can no longer act.
The game contains three types of characters:

Warrior: A warrior deals 15 points of damage to its target.

Healer: A healer restores 10 health points to its target, without exceeding the target's maximum health.

Mage: A mage has a certain number of spells. Each action consumes one spell and deals 25 points of damage. When the mage has no spells left, its action has no effect.

Damage cannot reduce health points below zero. An action targeting a dead character has no effect. A dead character cannot act.

Inventories

Each character has an inventory of items which, for simplicity, can be represented as `String`s (e.g., `"Potion"`, `"Sword"`, etc.).
When a living character deals damage to a target and that action reduces the target's health points to zero, the attacker collects **all the items** carried by the defeated character.
The collected items are appended to the attacker's inventory. The defeated character's inventory then becomes empty.
If the target survives, no items are transferred. An action against a character that is already dead has no effect and therefore cannot result in collecting any items.

Expected interface

You must design the classes needed to represent the characters and their relationships.
The following operations must be available through the common `Character` type:

- `name()`: Returns the character's name.
- `health()`: Returns the character's current health points.

- `maxHealth()`: Returns the character's maximum health points.
- `act(Character target)`: Performs the character's action on a target.
- `toString()`: Returns a `String` representing the character.
- `clone()`: Returns a copy of the character.

The constructors of the three character types must allow the following uses:

```
Warrior w = new Warrior("Arthur", 100);
Healer h = new Healer("Alice", 80);
Mage m = new Mage("Merlin", 60, 2);
```

The last parameter of the `Mage` constructor specifies its initial number of spells.
Characters must also provide the following operations.

- The method `addItem` adds an item into the inventory.
- The method `inventory()` provides access to the items owned by the character.

You may use a `List<String>` to represent the inventory.

Cloning characters

The method `clone()` must create a new character with the same state as the original character. In particular, the inventories of the original character and its copy must be independent: adding an item to the copy must not modify the inventory of the original character.

Combat example

The following program illustrates the expected behavior.

```
Character c = new Warrior("Arthur", 100);
Character d = new Healer("Alice", 80);

c.act(d);   // Arthur attacks Alice
System.out.println(d.health());  // 65

d.act(c);   // Alice heals Arthur
System.out.println(c.health());  // 100

Character e = c.clone();
System.out.println(e.health());  // 100

Character original = new Mage("Merlin", 60, 2);
original.addItem("Potion");

Character copy = original.clone();
copy.addItem("Key");

System.out.println(original.inventory().size()); // 1
System.out.println(copy.inventory().size());     // 2
```

Calls to `act()` must execute the behavior corresponding to the actual type of the character.

Task

Implement the classes required for this character system in Java. You are free to add any private or protected attributes and methods that you consider useful.
Write a `main` method to test your classes.

7 Polluter Robots (Optional)

A world is represented by a 2D matrix of boxes. Different kind of robots move in a world: *polluters* and *cleaners*.

- Polluters walk in a world and let greasy papers on boxes. There are two kinds of polluters: *jumpers* and *straighters* (these ones go straight). Each polluter leaves a greasy paper on each box where they stand.
- Cleaners remove greasy papers if there is one in the box where they stand. They walk in a *boustrophédon* style, i.e. they go through a whole line from right to left, then through a second line from left to right, and so on. There is one specific type of cleaners: *distracted cleaners* that remove only one paper out of two.

1. Define the class World with the following attributes, constructors and methods:
   - number of lines `nbL`, number of columns `nbC` and a boolean matrix `mat` with `nbL` lines and `nbL` columns (true if there is a greasy paper).
   - A constructor with the needed parameters and another one without parameters. The latter must create a 10×10 world without greasy papers.
   - Implement the method `public String toString()` that returns a string where empty boxes are represented with `.` and boxes with greasy papers with `x`.
   - `putsGreasyPaper(int x,int y)` : puts a greasy paper on the box (x,y).
   - `removesGreasyPaper(int x,int y)` : removes the greasy paper from the box (x,y).
   - `isDirty(int x, int y)` : tests if the box (x, y) contains a greasy paper.
   - `nbGreasyPapers()` : returns the number of greasy papers present in the world.

   The above methods must throw the appropriate exception if the parameters are not valid.
2. Define a class `TestWorld` with a method main for creating a 10×10 world. Test the different methods of the class `World`.
3. Define an abstract class `Robot` with the following fields and methods :
   - `posx`, `posy` : position of the robot in the world.
   - `m` : variable of type world
   - two constructors : `Robot(int x,int y,World m)` that creates a robot at position (x,y) and `Robot(World m)` that creates a robot at a random position (use `Math.random()`). The constructor `Robot(World m)` should call the other constructor.
   - `movesTo(int x, int y)` : to move to position (x, y).
   - `visit()` : abstract method (defined in sub-classes).
4. Define an abstract class `Polluter` that contains one method :
   - `pollute()` : puts a greasy paper in the box where the robot is.
   - constructor(s).
5. Define the class `Straighter` with the following fields and methods :
   - `startingColumn`, column that will be visited.

- constructor(s).
- `visit()` : overriding the abstract method of the class `Robot`. The robot must start at position `(0, startingColumn)` and move until the end of this column, one box at a time. A greasy paper is put in each visited box.

6. Define the class `Jumper` with the following fields and methods :
   - `deltax` : the number of boxes the jumper passes through.
   - constructors.
   - `visit()`, overriding the abstract method of the class `Robot` The robot must move from (0,0), then (1, `deltax`), then (2,2*`deltax`), then (3,3*`deltax`) (everything modulo `nbC`). The robot stops in the border of the world.
7. Define the class `Cleaner` with the following methods :
   - `clean()` : removes the greasy paper at the box where the robot is.
   - constructors,
   - `visit()` : overriding the abstract method of the class `Robot`. The expected behavior is to move from (0,0) straight to the end of the line, then next line in the other direction, and so on until the grid is fully visited. Greasy papers are removed as the robot visits the boxes.
8. Define the class `DistractedCleaner` with the following methods :
   - constructors.
   - `visit()` : redefinition of the method defined in `Cleaner`. The movement is the same but it removes only one paper out of two.
9. Create the class `TestRobot` to test the previous classes and methods.
10. (Optional) Currently, the method `visit` performs all the movements "at once" of the robot. Modify your classes to define a method `oneStep` that performs just one step at a time. Define also a class `Control` that picks randomly one of the robots and makes it "move" one step.